

Topic 3

Intersystem communication

Agenda

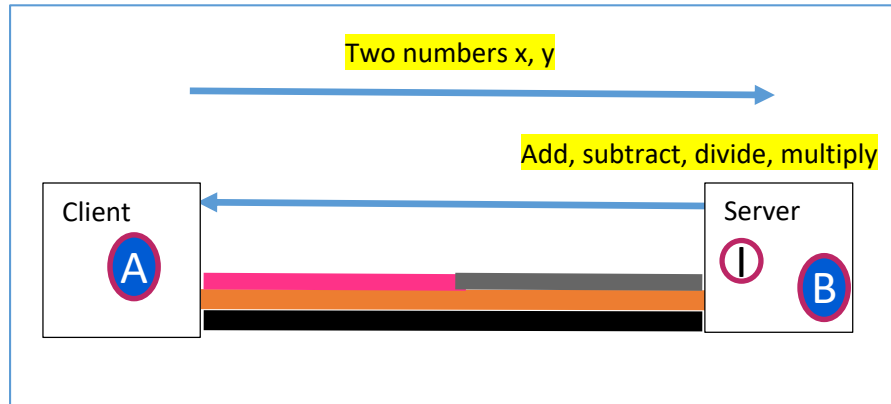
- Overview of the previous lecture
- Introduction
- Distributed applications
 1. Features of distributed environment
 2. Distributed applications: advantages and challenges
 3. Distributed architecture: DOA, SOA
- ~~■ Network programming~~
 - ~~1. Sockets and streams~~
 - ~~2. Accessing URL~~
- ~~■ CORBA~~
- Remote Message Invocation
- ~~■ Web services~~
- ~~■ Messaging services (self study)~~
- ~~■ Important resources~~
- Summary

Announcement

- Test 2: Tentatively 4 May 2026
- Test 3: To be decided

❑ **Part 3: Remote Method Invocation**

Remote Method Invocation (RMI)

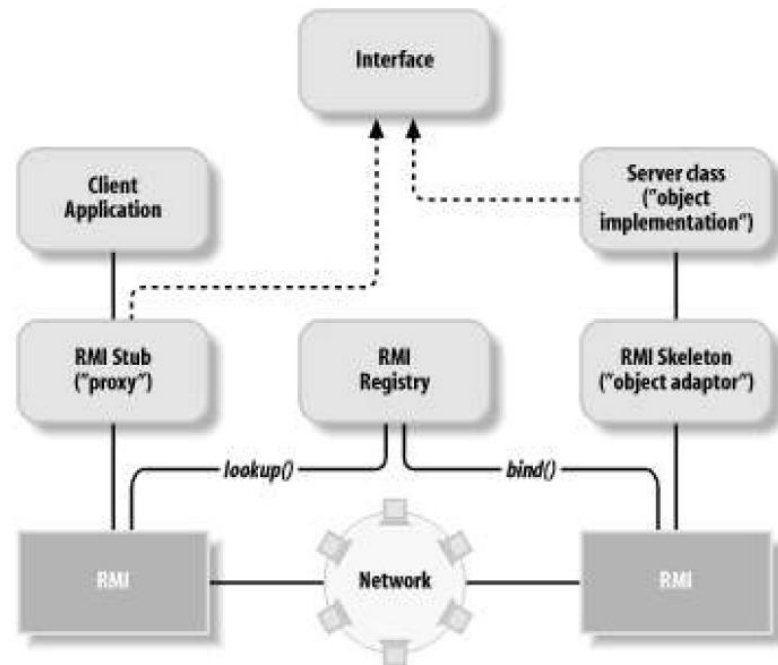


- ❑ In RMI, a local object invokes the methods of a remote object as if it is a local object
- ❑ The figure above is a simple representation
 1. A and B are distributed objects on client and server
 2. A accesses methods of the remote object B over the network
 3. I is a remote interface (implemented by the remote class of B)
- ❑ The main parts of RMI:
 1. Stub (pink) and Skeleton (gray) layer:
 - a. Stub marshals (or “packages”) and sends the request to B; demarshals (i.e., unpacks) response from B
 - b. Skeleton (gray) demarshals the request; handles requests to B; marshals response to A
 2. Remote reference layer (brown): handles references to the remote object
 3. Transport layer (black): handles the communication (i.e., request and response between client and server)

Creating Java RMI application

❑ A detailed view

- A remote interface “advertises” methods of the remote object to the outside world
- Server creates a registry and binds an instance of the remote object in the registry object to a **unique name** (i.e. reference)
- Client “looks up” a specific reference in the registry
- Client calls the remote methods of the reference to perform functions

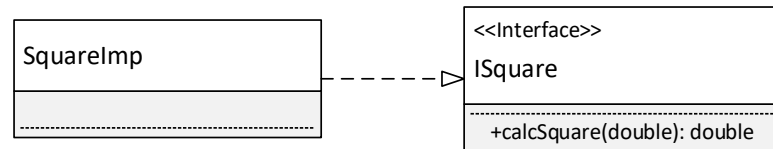


Java RMI Application: Example

- ❑ The Java `java.rmi` package supports RMI programming
- ❑ Steps
 1. Write the code for a remote interface
 - a. This should `extend` a **remote interface** (`Remote`)
 - b. Each method throws an exception (`RemoteException`)
 2. Write code for the remote object (i.e., implementation class of remote interface. This class
 - a. Implements the remote interface above
 - b. Extends a superclass `UnicastRemoteObject` (which ensures each remote object is unique)
 - c. Must contain a constructor (`to create remote objects`)
 - d. `Each method (including constructor)` also declares `RemoteException` (as in the interface above)
 3. Write code for the server
 - a. Create remote object(s) of the class in 2
 - b. Create registry
 - c. Bind (i.e., register) the instance of remote object with a unique name (i.e., referencing)
 4. Create the client; this class :
 - a. Creates a “stub” to look for the reference to the remote object in 3c
 - b. Uses the “stub” to call methods on the remote object

Java RMI Application: Example

- ❑ You are required to write a distributed application which consists of:



1. The remote interface `ISquare` (contains a method `calcSquare(double)`, which returns the square of a number)
2. The class `SquareImp` implements the remote interface
3. The class `ServerTester` (which creates, registers and binds instance (`obj`) of `SquareImp`). Use “`square`” (for remote reference) and port `2026`.
4. The class `ClientTester` calls `calcSquare(double)` with an appropriate value and displays the answer. A typical output is:

The square of 4 is 16

Java RMI Application: Example 1

❑ This program consists of four parts (**the server part consists of 1, 2 and 3**)

1. A **remote interface** (*ISquare.java*) which declares a method **calcSquare(double)**

```
ISquare.java
I ISquare
  calcSquare(double) : double
```

2. A class *SquareImp.java* which implements *ISquare.java* and implements **calcSquare(double)**

```
SquareImp.java
G SquareImp
  SquareImp()
  calcSquare(double) : double
```

3. A class *ServerTester.java* which **registers** and **binds** remote objects of *ISquareImp.java*

```
ServerTester.java
G ServerTester
  main(String[]) : void
```

4. A class *ClientTester.java* which creates a stub to invoke the method of the remote object in 3

```
ClientTester.java
G ClientTester
  main(String[]) : void
```

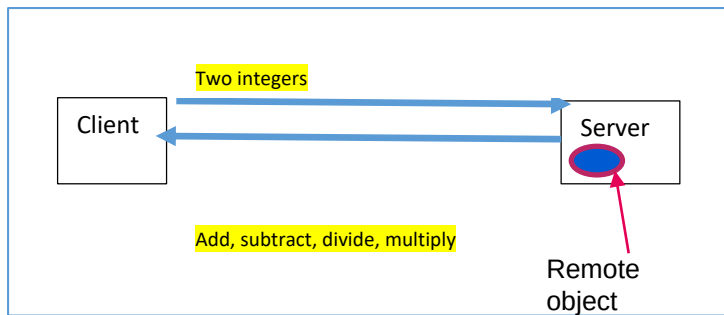
❑ **NOTE: use the descriptions on slide 8**

Java RMI Application: Example 1

- ❑ See the complete example (attached)

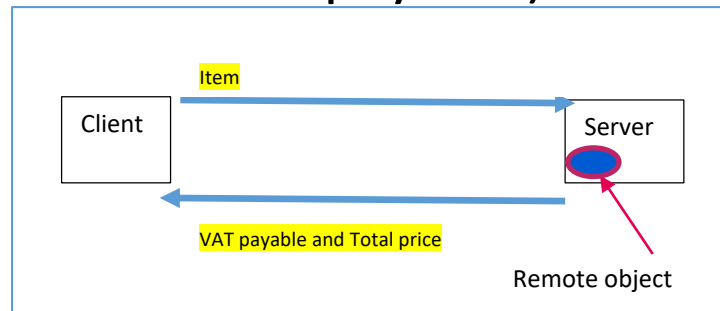
Java RMI Application: Exercise 1

- ❑ You're required to write a RMI distributed application which allows a client to request the methods of a remote object to calculate the sum, difference and square of two integers and display the results on the screen.



Java RMI Application: Example 2

- ❑ You're required to write a distributed application which allows a client to access the methods of the interface `IPriceCalculator.java` of a remote object `PriceCalculator.java`. `IPriceCalculator.java` declares VAT (15%) as well as the methods `calcVAT(Item)` and `calcTotalPrice(Item)` which return the VAT payable (as 15% of the price of an item) and the total price (as the price of an item and VAT payable).



- ❑ Note: Item object must be “**serialized**” and transmitted over the network.

Comments and feedback

?

Next.....

- Scripting (with JSP)